TP 1 : Créer un émetteur simple qui gère des paramètres.

1. Créez une instance de `EventEmitter` nommée `monReveil`.
2. Créez un écouteur pour l'événement `sonnerie`. Cet écouteur doit recevoir une heure (ex: "07h00") et afficher : "Debout ! Il est [heure]".
3. Émettez l'événement deux fois avec des heures différentes.

TP 2 : Utiliser `.once()` et les erreurs.

1. Créez un émetteur qui simule un scanner de badge.
2. Utilisez `.once('connexion', ...)` pour afficher "Système initialisé" la première fois qu'un badge est passé.
3. Créez un écouteur pour l'événement `'error'`.
4. Émettez un événement `scan` :
    a. Si le nom du badge est "Inconnu", émettez un événement `'error'` avec un message de menace.
    b. Sinon, affichez "Accès autorisé".

TP 3 : Créer un fichier, écrire dedans, lire son contenu, ajouter du contenu sans écraser l'existant, puis organisateur de dossiers.

1. Importez `fs` (version promises).
2. Créez un fichier nommé `bienvenue.txt` contenant le texte : *"Hello Node.js !"*.
3. Lisez ce fichier et affichez son contenu dans la console.
4. Créez une fonction ajouterLog(message).
5. À chaque appel, cette fonction doit ajouter une ligne au fichier `logs.txt` avec la date actuelle et le message (ex: `[2026-10-27] Serveur démarré`). Utilisez la méthode `appendFile`.
6. Vérifiez si un dossier nommé Archives existe. Sinon, créez-le.
7. Renommez (déplacez) le fichier bienvenue.txt (créé à l'exo 1) vers Archives/ancien_message.txt.
8. Supprimez le fichier d'origine s'il existe encore.

TP 4 : Streams.

1. Fournir l'exécution étape par étape dans la pile d'appels (call stack) des codes de la diapositive 78.
2. D'où le stream lit les données dans ces codes?
3. Modifier ce code pour lire le contenu d'un fichier réel à la place.
4. Modifier ce code pour lire le contenu par le clavier.
5. Fournir l'exécution étape par étape dans la pile d'appels (call stack) des codes de la diapositive 78 en utilisant l'évenement 'readable' à la place de 'data'.

TP 5 : Créer un script qui copie un fichier volumineux d'un dossier vers un autre, tout en affichant la progression en temps réel (en octets) dans la console.

Consignes :

1. Utilise `path.join` pour définir un fichier source (un gros `.txt`) et un fichier de destination dans un dossier nommé `/copies`.
2. Crée un **ReadStream** et un **WriteStream**.
3. Écoute l'événement `data` sur le flux de lecture pour :
   a. Compter le nombre d'octets reçus à chaque "chunk" (morceau).
   b. Afficher dans la console : `"Chargement : [nombre] octets reçus..."`.
4. Utilise la méthode `.pipe()` pour envoyer les données vers le flux d'écriture.
5. Écoute l'événement `finish` sur le flux d'écriture pour afficher : `"Copie terminée avec succès !"`.